Rayos X, radiografías y gVirtualXray como su gemelo digital

La radiografía convencional es esencial porque permite visualizar rápidamente la estructura interna de objetos y del cuerpo humano de forma no lesiva a la par que económica. Su simulación virtual (aunque no entren en juego ni personas ni radiación real) resulta también muy útil en contextos como educación, planificación clínica, diseño de dispositivos, etc., ofreciendo un entorno seguro y flexible para experimentar y aprender.

En el ámbito clínico, los rayos X *convencionales* siguen siendo la técnica de imagen médica más utilizada a nivel mundial desde que fueron descubiertos por Wilhelm Röntgen en 1895. Qué duda cabe que una radiografía tradicional se manifiesta fundamental para la detección de fracturas, alteraciones óseas y otras múltiples patologías. También tiene un alto valor a nivel industrial (controles de seguridad y/o calidad). Su ventaja radica en que combina una alta resolución espacial con tiempos de adquisición muy cortos y dosis de radiación bajas en comparación con modalidades más complejas (y más ionizantes y molestas) como la tomografía computarizada (TAC), la fluoroscopia o la tomosíntesis. Su simplicidad, eficacia diagnóstica y disponibilidad global la mantienen como una herramienta indispensable tanto dentro como fuera del ámbito de la evaluación clínica. En este artículo vamos a comenzar repasando los fundamentos físicos (y técnicos) de esta modalidad radiológica para luego explorar las variantes y ventajas de su *gemelo digital*: el radiograma virtual.

Funcionamiento general de un sistema de rayos X

Un sistema de rayos X común funciona generando radiación electromagnética de alta energía (del orden de las decenas-centenas de keV) en un tubo de vidrio a muy baja presión. En el mismo, un haz de electrones es acelerado y acaba *impactando* (más bien siendo *desacelerado*, como luego veremos) contra un ánodo metálico. Esta interacción produce *oleadas* de fotones X con un histograma de intensidad-frecuencia muy característico (véase Figura 1). En este sentido el conjunto de ánodo-detector se comporta como una suerte de cámara estenopeica (*pinhole*) donde, curiosamente, la focal depende fuertemente de la geometría y posición relativa [1].

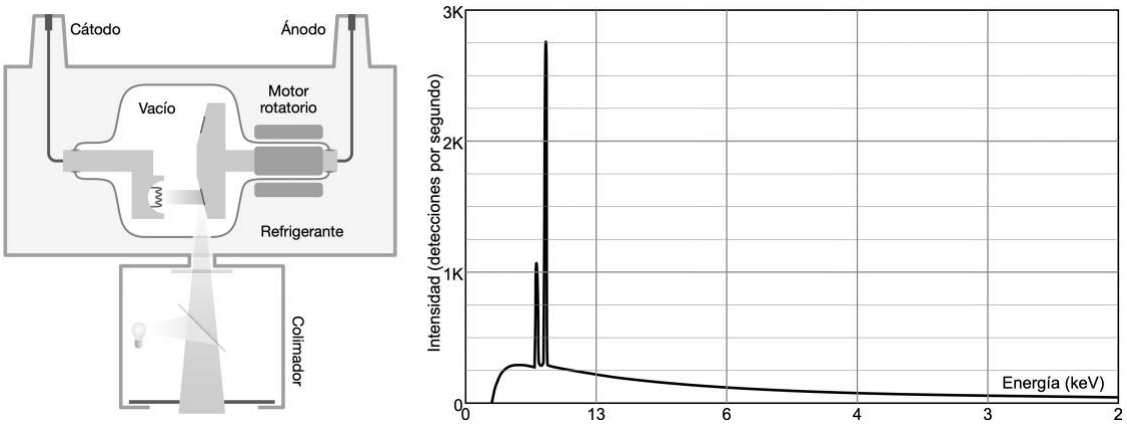

Figura 1. Partes de un tubo Röntgen (izquierda) y espectro emitido por un ánodo de rodio, operado a 60 kVp (derecha). La curva continua corresponde al bremsstrahlung y los picos son las líneas K características del Rh-45.

Estos *rayos Röntgen*, conocidos también como *radiación de frenado* o *bremsstrahlung*, atraviesan el cuerpo u objeto a estudiar y son *atenuados* en su camino en función de la densidad y composición de los tejidos o materiales. Las estructuras densas (como los huesos) absorben más radiación, mientras que tejidos *blandos* (-100 a +100 unidades Hounsfield) la dejan pasar en mayor medida. La radiación transmitida llega a un detector (ya sea una película radiográfica basada en una emulsión de haluros de plata, o un detector digital), que convierte la intensidad de rayos recibida en una proyección bidimensional. En esta representación, las zonas más radio-opacas aparecen claras y las más *transparentes*, oscuras. En la Figura 2 se muestran varias radiografías y su composición *dual* para resaltar tejidos blandos o huesos según sea necesario (densitometría).

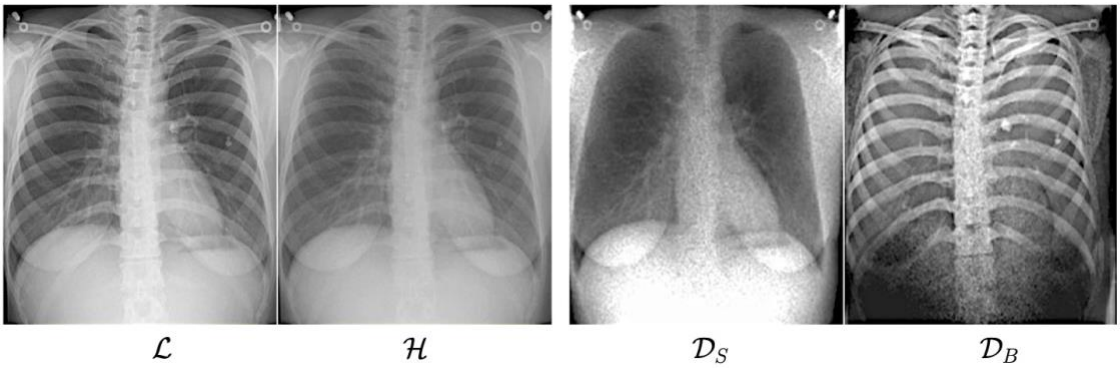

Figura 2. Ejemplo de imágenes de rayos X duales para resaltar tejidos blandos (D_S) o huesos (D_B) a partir de una imagen de baja energía (L) generada a 60 kVp y una radiografía de alta energía (H) producida a 120 kVp.

En radiología convencional, un técnico de rayos X puede ajustar la intensidad y energía del haz modificando el *kilovoltaje pico* (kVp) aplicado entre el cátodo y el ánodo del tubo. En la práctica clínica, la radiografía de una mano suele requerir 60 kVp, un hombro 70 kVp (tensión a partir de la cual se hace necesario el uso de una rejilla antidispersión para reducir el *scatter*), un abdomen aproximadamente 80 kVp, y un tórax alrededor de 110 kVp. Estas variaciones permiten optimizar la penetración del haz y la calidad de la imagen según la región anatómica a explorar. El ánodo tiene forma de disco rotatorio para disipar el calor generado cuando los electrones impactan en su superficie, ya que solo una pequeña fracción de la energía se convierte en radiación X.

Los rayos X en la materia

Por descontado, el *mitigamiento* de los haces Röntgen al pasar por la materia es mucho más complejo con respecto a cómo lo hemos descripto en el apartado anterior. La atenuación de los rayos X al atravesar un material se describe mediante la ley de Beer–Lambert [2], la cual modela cómo la intensidad disminuye exponencialmente con la distancia recorrida y las propiedades del material. Si I_0 es la intensidad inicial (número de fotones N_0 multiplicado por su energía E_j) e I la transmitida tras atravesar una longitud L, se cumple:

$$I = I_0\, e^{-\mu L}\,,$$

donde μ es el coeficiente de atenuación lineal (en cm^{-1}), que depende de la densidad ρ, el número atómico Z del material y, nuevamente, de la energía E_j de los fotones incidentes. En un objeto heterogéneo (con distintas componentes/capas, cada uno con su ρ_i), la expresión se generaliza como:

$$I = I_0 \exp\left(-\sum_i \mu_i\,(E, \rho_i, Z_i)\, L_i\right),$$

donde la suma recorre todos los materiales (cada uno de una longitud L_i) atravesados por un haz monocromático (de energía E_j). Esto explica por qué en una radiografía los huesos (alta μ) se muestran en tonos claros y los tejidos blandos (baja μ) más oscuros. Pero claro, la emisión Röntgen nunca es monocromática, sino que tiene un espectro de energías ($E_1, E_2, \ldots, E_j, \ldots$) muy característico (como el de la Figura 1). Entonces, la expresión anterior se transforma en:

$$I = \sum_j E_j\; N_0(E_j) \exp\left(-\sum_i \mu_i\,(E_j, \rho_i, Z_i)\, L_i\right),$$

donde, otra vez, $N_0(E_j)$ es el número de fotones incidentes a esa energía y $\mu_i(E_j, \rho_i, Z_i)$ es el coeficiente de atenuación lineal dependiente del material (con número atómico Z_i) y la energía E_j.

Radiografías artificiales

La simulación de rayos X y radiografías es especialmente interesante porque permite recrear de manera realista el proceso de adquisición de imágenes sin necesidad de exponer a pacientes, animales o materiales a radiación ionizante, lo que reduce riesgos y costes. Además, este gemelo digital brinda un entorno controlado donde se pueden variar parámetros del sistema (geometría, energía, materiales, detectores, etc.) para optimizar protocolos clínicos o de investigación, diseñar nuevos equipos y evaluar su rendimiento antes de fabricarlos. También es clave en educación, al ofrecer a estudiantes y profesionales la posibilidad de experimentar con radiografías virtuales de forma interactiva. En el ámbito de la inteligencia artificial, las imágenes radiográficas *sintéticas* permiten entrenar algoritmos de detección, segmentación o clasificación.

Dependiendo de la técnica de simulación del haz de rayos y del tipo de *target virtual* (con lo que interaccionan), tenemos distintos tipos radiografías virtuales. Empecemos repasando las distintas alternativas para simular rayos X, cada una con un diferente equilibrio entre precisión, objetivo y velocidad.

Rayos X simulados

Por un lado, los métodos Monte Carlo, como los usados en los famosos paquetes Geant4 [3] o Gate [4], son los más realistas, ya que modelan estadísticamente la interacción física de cada fotón con la materia, incluyendo dispersión y efectos cuánticos, pero requieren mucho tiempo de cálculo. Por otro, los enfoques basados en la ley de Beer–Lambert son más rápidos: pueden implementarse mediante *ray casting* o *ray tracing*, siguiendo el recorrido de cada rayo a través de la geometría. También existen métodos híbridos que combinan Beer–Lambert con modelos simplificados de dispersión (*rasterización*) y/o respuesta del detector para mejorar el realismo sin sacrificar demasiado el rendimiento.

En gráficos por ordenador, el ray casting se emplea como un método básico en el que desde el sensor de la cámara se *lanza un rayo* por cada píxel x_i, y_j para encontrar el primer objeto intersecado, lo que lo hace útil para generar imágenes sencillas sin efectos de iluminación compleja. El ray tracing, en cambio, lleva esta idea más allá: tras la primera intersección, sigue propagando rayos secundarios para simular fenómenos ópticos como sombras, reflejos, refracciones o dispersión de la luz, logrando imágenes altamente realistas a costa de una mayor demanda computacional.

En la simulación de radiografías *sobra*, en principio, con la aproximación de ray casting. En este escenario, la radiación *parte* desde un ánodo virtual y se calculan únicamente las intersecciones con los objetos para determinar las longitudes de trayecto y aplicar Beer–Lambert, lo que permite obtener proyecciones rápidas y suficientes para reproducir la absorción principal del haz. En este caso, el parámetro más relevante y complicado de calcular es, precisamente, cada una de las distancias atravesadas (L_i).

A modo de ejemplo sencillo, el siguiente código Python computa la radiografía virtual de una esfera de densidad homogénea mediante ray casting y teniendo en cuenta la física discutida hasta ahora.

```
import numpy as np
from math import exp, sqrt; import matplotlib.pyplot as plt
src = np.array([0, 0, -10]); det_z = 5; res = 400; pitch = 0.0125; half = pit
ch * (res - 1) / 2 # Geometría del entorno RX
R = 1.25; mu = 0.365 # Esfera examinada (densidad constante)
E, N = 80.0, 15000; img = E * N * np.ones((res, res)) # Haz y detector

def intersect_sphere(o, d):
    b = 2 * np.dot(o, d)
    c = np.dot(o, o) - R**2
    disc = b*b - 4*c
    return None if disc < 0 else abs(sqrt(disc))

for j in range(res):
    for i in range(res):
        p = np.array([-half + i*pitch, -half + j*pitch, det_z])
        d = (p - src) / np.linalg.norm(p - src)
        L = intersect_sphere(src, d)
        if L:
            img[j, i] *= exp(-mu * L)

plt.imshow(img, cmap="gray"); plt.show()
```

El resultado de este proceso es el que podemos ver en la Figura 3.

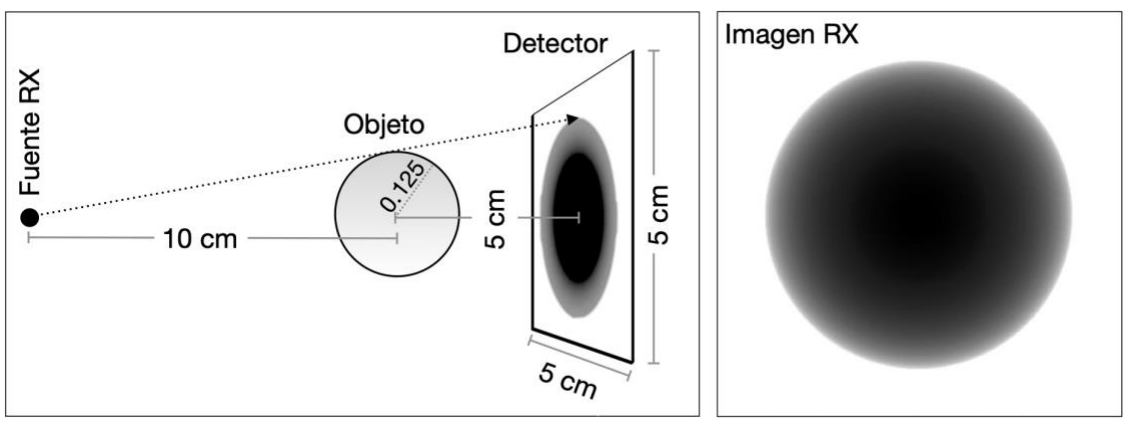

Figura 3. Disposición del experimento mental descrito en el código Python anterior e imagen RX virtual generada.

Si además el target simulado es una *malla* (o conjunto de mallas) poligonal, aún se puede *exprimir* más esta velocidad mediante métodos de rasterización de triángulos en GPU. Eso es precisamente lo que vamos a discutir a continuación: cómo la descripción digital de los objetos analizados puede influir en la rapidez de generación de *röntgengramas*.

Targets virtuales

Dependiendo del blanco virtual usado, podemos estar hablando de una radiografía reconstruida digitalmente (*Digitally Reconstructed Radiograph*) o de la proyección geométrica de una figura tridimensional mallada.

Las primeras son radiografías sintéticas generadas a partir de una cuadrícula tridimensional, la cual suele ser, típicamente, el volumen obtenido previamente desde un TAC. La imagen radiográfica final se obtiene proyectando los valores de atenuación del volumen (*vóxeles*) a lo largo de trayectorias que simulan el paso de rayos X y calculando la intensidad transmitida en cada píxel (tal y como hemos visto previamente). El resultado es una imagen 2D que reproduce la apariencia de una radiografía real desde una vista (ángulo, distancia, etc.) específica. En este caso, cada vóxel contiene el coeficiente de atenuación del material i, es decir: $\mu(x, y, z, \rho_i, E_j)$. La intensidad transmitida se calcula integrando la atenuación vóxel a vóxel a lo largo del trayecto de cada rayo, lo que permite un modelado continuo, detallado, fiel y de calidad de estructuras internas, pero con un alto costo computacional y de memoria. Dicho de otra manera, resulta laborioso derivar cada L_i.

En cambio, cuando se usa un *mesh* 3D, los objetos se *mallan*. De esta manera, los volúmenes se representan por sus superficies externas y los rayos solo interactúan con los triángulos de esas fronteras. Esto simplifica el cálculo de cada longitud de trayecto (L_i) dentro de cada material mediante sencillas intersecciones de entrada y salida (o mediante técnicas de raster). En este escenario, el único caso de aplicación es el proyecto gVirtualXray (Figura 4), que veremos a continuación.

gVirtualXray

gVirtualXray [5] es framework escrito en C++ y que utiliza geometrías triangulares y la rasterización en GPU para estimar las trayectorias de los rayos, aplicando igualmente la ley de Beer–Lambert, pero con una velocidad muy superior. Con el proceso de rasterización, el sistema proyecta cada triángulo sobre el plano del detector y determina qué píxeles quedan cubiertos por él (*test de cobertura*). Luego, para cada píxel, se calculan valores interpolados a partir de los vértices. Esta información se acumula en una *memoria de longitudes* atravesadas (o *L-buffer*). Este enfoque es mucho más rápido y eficiente en GPU, aunque menos preciso si el objeto simulado representa una realidad más compleja desde el punto de vista estructural. Dicho de otra manera, gVirtualXray no usa algoritmos de intersección rayo–triángulo, sino de rasterización.

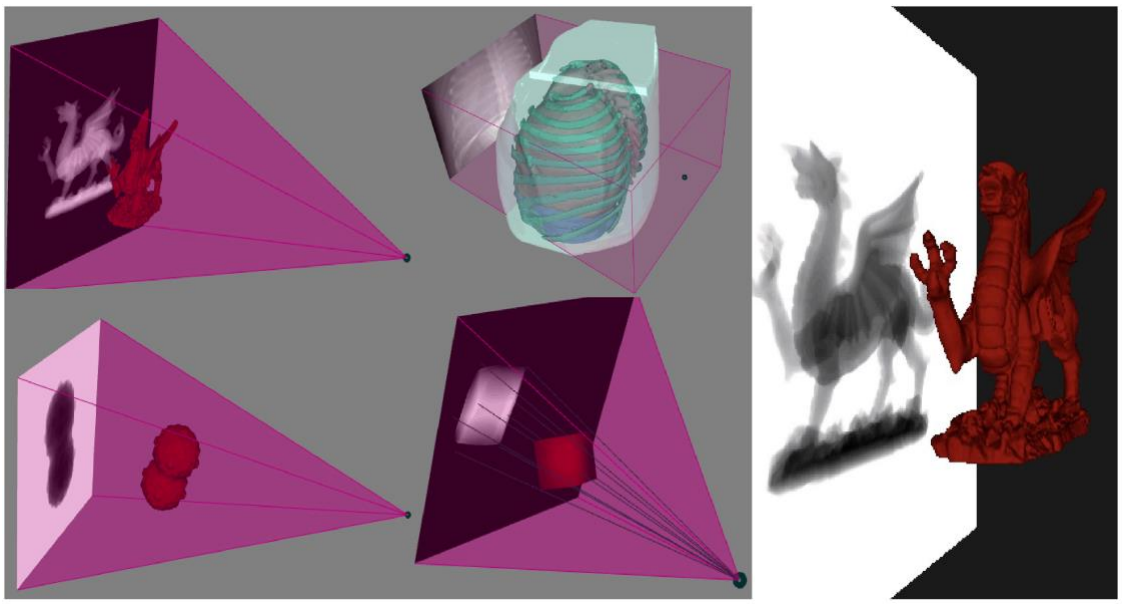

Figura 4. Proyecciones radiográficas de mallas 3D con gVirtualXray.

Para finalizar, ya solo queda aplicar Beer-Lambert, pero este cálculo es prácticamente inmediato. La gran eficiencia de gVirtualXray proviene de aprovechar bien los *pipelines de rendering* para obtener trayectorias de rayos de forma determinista y masivamente paralelizable. Con un poco más de detalle, gVirtualXray utiliza el algoritmo descrito en [6], el cual combina proyección en perspectiva y un procesamiento tipo *z-buffer*. Cada triángulo se proyecta desde la fuente S al detector y, para cada píxel P cubierto, el punto de intersección se obtiene a lo largo del rayo:

$$\mathbf{I} = \mathbf{S} + b_p(\mathbf{P} - \mathbf{S}), \qquad b_p = \frac{\mathbf{n} \cdot \mathbf{S} + d}{\mathbf{n} \cdot (\mathbf{S} - \mathbf{P})},$$

donde $\mathbf{n}$ es la normal del triángulo. El signo de $\mathbf{n} \cdot \overrightarrow{AS}$ distingue entradas y salidas, y la longitud de trayectoria se calcula como:

$$L_p = |\mathbf{SP}| \sum_i - \operatorname{sgn}(N_i)\, b_{p,i}\,.$$

Esta formulación permite *acumular* eficientemente L_p para simular la atenuación radiográfica.

Un gemelo digital eficiente y muy exacto

gVirtualXray ha sido validado mediante una comparación exhaustiva entre sus resultados simulados y datos obtenidos con simulaciones Monte Carlo y radiografías experimentales reales con fantomas antropomórficos [7]. En los estudios de validación [8], se evaluaron métricas como la intensidad promedio, el contraste relativo y el error de atenuación,

mostrando diferencias menores al 3 % respecto a los modelos Monte Carlo, pero con tiempos de cálculo hasta 10^5 veces más rápidos (Figura 5).

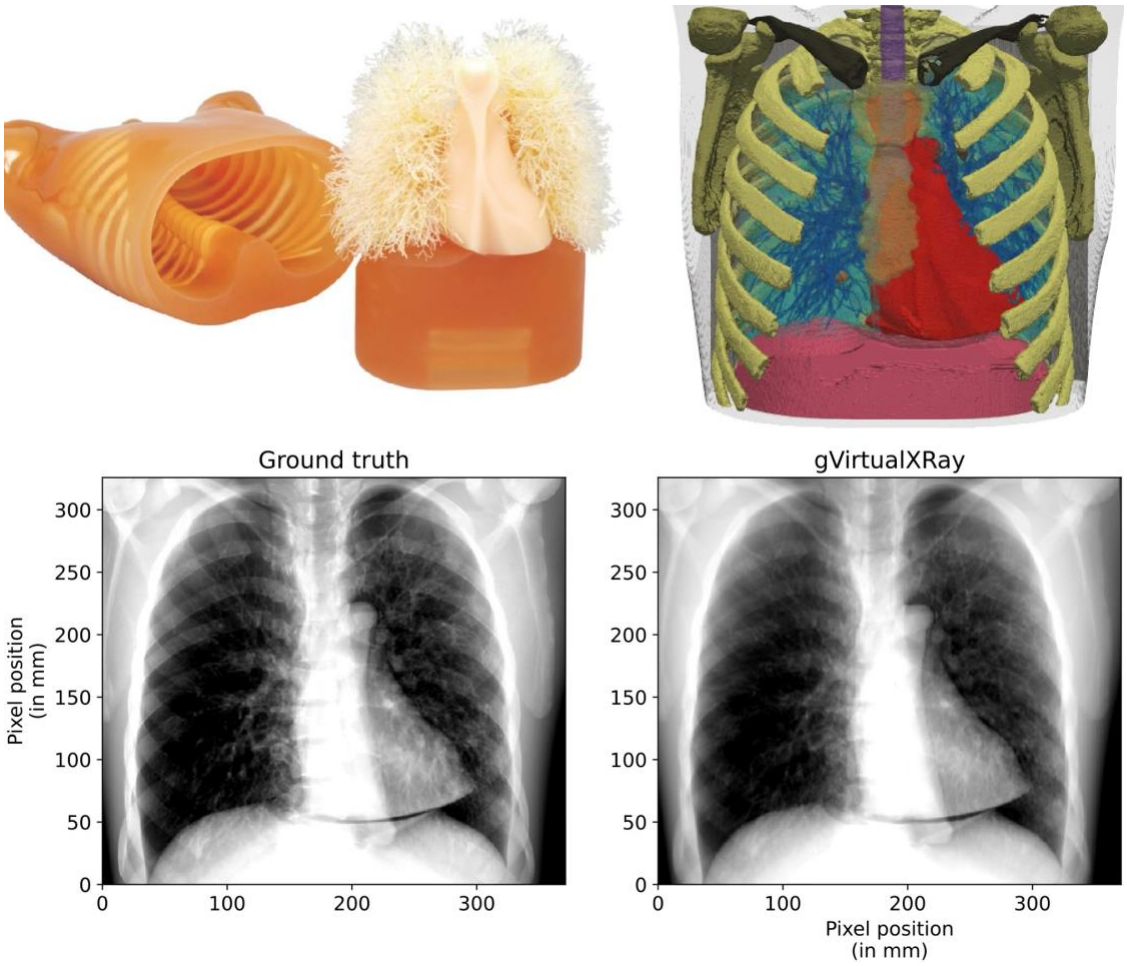

Figura 5. Comprobación visual de la bondad de gVirtualXray para simular la radiografía de un fantoma antropomórfico del cual se conoce su estructura 3D (gemelo digital).

gVirtualXray como framework

gVirtualXray ofrece una serie de APIs multiplataforma que facilitan su integración en distintos entornos y lenguajes de programación. Su núcleo está escrito en C++, lo que garantiza alta eficiencia y portabilidad, pero además cuenta con *bindings* para Python, C#, Java, Ruby, R, Tcl y GNU Octave, permitiendo a investigadores, ingenieros y educadores usarlo sin necesidad de trabajar directamente con código de bajo nivel. Estas APIs exponen funciones para cargar geometrías (mallas o volúmenes), definir materiales y fuentes, configurar detectores y parámetros espectrales, y ejecutar simulaciones tanto monoespectrales como policromáticas. En Python (su API más utilizada) se distribuye como el paquete gVXR en PyPI, con una estructura orientada a objetos y soporte completo para visualización interactiva y procesamiento de resultados, lo que convierte a

gVirtualXray en una herramienta versátil y accesible desde múltiples entornos científicos y de desarrollo.

A modo de ejemplo, el siguiente código realiza una tarea muy parecida al visto más arriba, pero gVirtualXray posibilita una redacción más limpia y entendible.

```
# Instalar primero gVirtualXray con: pip install gvxr
import numpy as np
from tifffile import imwrite
from matplotlib import pyplot as plt
from gvxrPython3 import gvxr;

gvxr.createOpenGLContext()
gvxr.setDetectorPosition(0, 0, 5, "cm") # Definimos el detector
gvxr.setDetectorUpVector(0, 1, 0)
gvxr.setDetectorNumberOfPixels(400, 400)
gvxr.setDetectorPixelPitch(0.0125, 0.0125, "cm")
gvxr.setSourcePosition(0, 0, -10, "cm") # Definimos la fuente y el espectro
gvxr.usePointSource()
gvxr.setMonoChromaticPerPixelAtSDD(80, "keV", 15000)
gvxr.makeSphere("sphere", 50, 50, 1.25, "cm") # Definimos el objeto a examina
r
gvxr.addPolygonMeshAsInnerSurface("sphere")
gvxr.setElement("sphere", "C")
# Calculamos la radiografía
xray_image = np.asarray(gvxr.computeXRayImage()) / gvxr.getUnitOfEnergy("keV")
plt.imshow(xray_image, cmap="gray")
plt.show()
```

Conclusiones

En este artículo hemos discutido cómo la simulación de radiogramas constituye un gemelo digital valioso de la radiografía convencional, combinando fundamentos físicos bien establecidos con técnicas computacionales modernas. En concreto, gVirtualXray puede emplearse en educación [9] como una herramienta interactiva para enseñar los principios físicos, geométricos y tecnológicos de la formación de imágenes por rayos X sin necesidad de equipos radiológicos reales ni exposición a radiación. También permite a los estudiantes visualizar cómo influyen parámetros como la energía del haz, el espesor o composición de los materiales, la posición de la fuente o la geometría del detector en la apariencia final de una radiografía. Gracias a su ejecución en tiempo real y su compatibilidad con lenguajes como Python, puede integrarse fácilmente en laboratorios virtuales [10] o entornos de aprendizaje (Figura 6), favoreciendo la experimentación [8], la comprensión conceptual y la simulación segura de escenarios clínicos, industriales e incluso veterinarios [11].

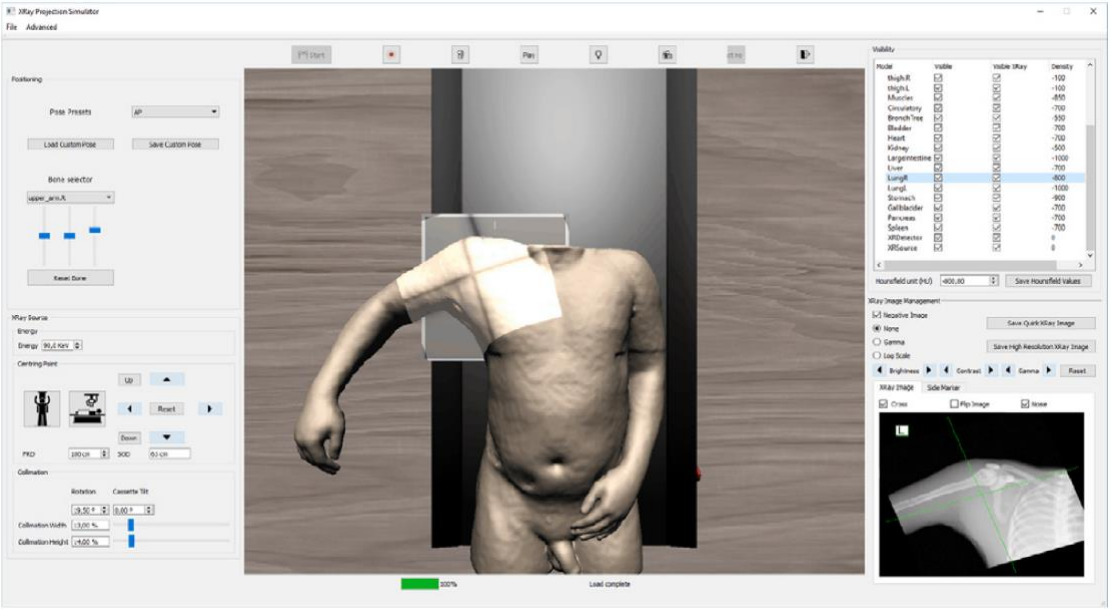

Figura 6. Software educativo [12] basado en gVirtualXray para prácticas en radiología.

Autores

Foto	Autor
	Alberto Corbi – Escuela Superior de Ingeniería y Tecnología, Universidad Internacional de La Rioja (UNIR).
	Franck P. Vidal – UKRI-STFC Scientific Computing, Daresbury Laboratory & School of Computer Science & Engineering, Bangor University.
	Francisco Albiol – Instituto de Física Corpuscular (CSIC-UVEG).
	Alberto Albiol – Escuela Técnica Superior de Ingeniería de Telecomunicación, Universidad Politécnica de Valencia.
	Fabio R. Llorella – Escola Politècnica Superior, Universitat de Girona.

Referencias

[1] F. Albiol, A. Corbi, and A. Albiol, “3D measurements in conventional X-ray imaging with RGB-D sensors,” *Medical engineering & physics*, vol. 42, pp. 73–79, 2017.

[2] D. F. Swinehart, “The Beer-Lambert law,” *Journal of chemical education*, vol. 39, no. 7, p. 333, 1962.

[3] S. Agostinelli, J. Allison, K. al Amako, J. Apostolakis, H. Araujo, P. Arce, M. Asai, D. Axen, S. Banerjee, G. Barrand, and others, “Geant4—a simulation toolkit,” *Nuclear instruments and methods in physics research section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, vol. 506, no. 3, pp. 250–303, 2003.

[4] D. Strulab, G. Santin, D. Lazaro, V. Breton, and C. Morel, “GATE (Geant4 Application for Tomographic Emission): a PET/SPECT general-purpose simulation platform,” *Nuclear Physics B-Proceedings Supplements*, vol. 125, pp. 75–79, 2003.

[5] A. Sujar, A. Meuleman, P.-F. Villard, M. Garcia, and F. P. Vidal, “gVirtualXRay: Virtual X-ray imaging library on GPU,” in *Computer graphics and visual computing*, 2017, pp. 61–68.

[6] N. Freud, P. Duvauchelle, J.-M. Létang, and D. Babot, “Fast and robust ray casting algorithms for virtual X-ray imaging,” *Nuclear Instruments and Methods in Physics Research Section B: Beam Interactions with Materials and Atoms*, vol. 248, no. 1, pp. 175–180, 2006.

[7] J. L. Pointon, T. Wen, J. Tugwell-Allsup, J. M. Létang, and F. P. Vidal, “gVirtualXray (gVXR): Simulating X-ray radiographs and CT volumes of anthropomorphic phantoms,” *Software Impacts*, vol. 16, p. 100513, 2023.

[8] F. P. Vidal and P.-F. Villard, “Development and validation of real-time simulation of X-ray imaging with respiratory motion,” *Computerized Medical Imaging and Graphics*, vol. 49, pp. 1–15, 2016.

[9] A. Sujar, G. Kelly, M. García, and F. P. Vidal, “Interactive teaching environment for diagnostic radiography with real-time X-ray simulation and patient positioning,” *International Journal of Computer Assisted Radiology and Surgery*, vol. 17, no. 1, pp. 85–95, 2022.

[10] I. T. Mitchell, J. M. Létang, L. M. Evans, and F. P. Vidal, “WebCT–OpenSource web-based GUI for real-time X-ray simulation,” *Tomography of Materials and Structures*, vol. 8, p. 100057, 2025.

[11] X. Xu, A. Puggioni, D. Kilroy, and A. G. Campbell, “Mixed Reality and Real-Time X-Ray Simulation in Vet Radiography Training: A User-Centered Comparative Study,” in *2025 IEEE conference virtual reality and 3D user interfaces (VR)*, 2025, pp. 12–22.

[12] A. Sujar, G. Kelly, M. García, and F. Vidal, “Projectional Radiography Simulator: an Interactive Teaching Tool,” in *Computer graphics and visual computing (CGVC)*, 2019.